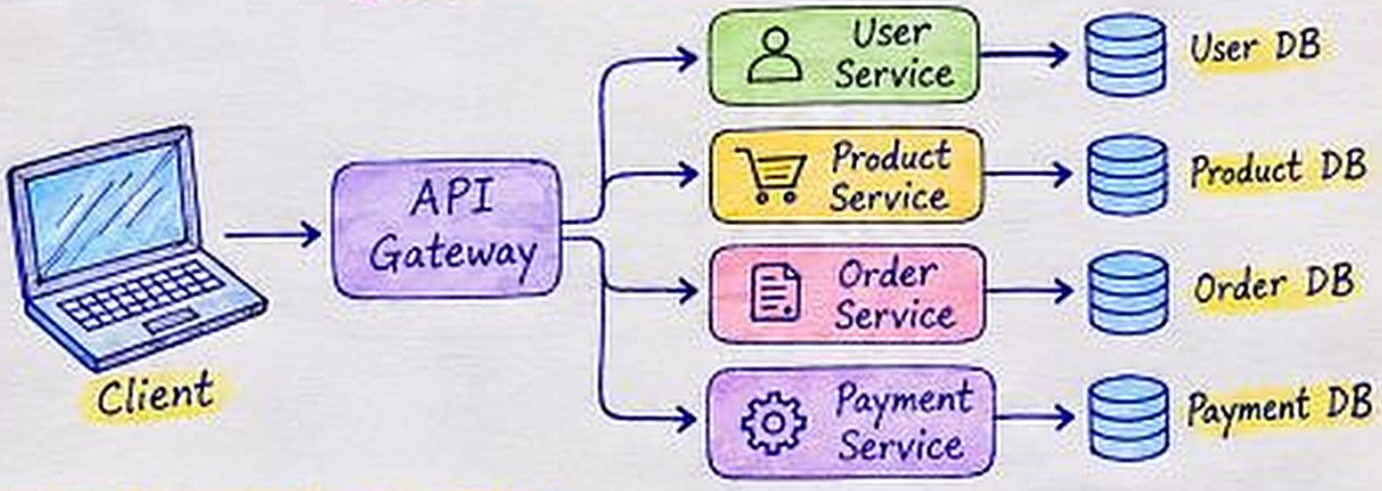
Microservices

Complete Handbook

From Basics → Advanced → System Design

Learn how modern applications are built, scaled & deployed using Microservices.



What You'll Learn

01 Fundamentals
02 Architecture
03 Service Communication
04 API Gateway
05 Database & Data Consistency
06 Kafka & Event-Driven
07 Saga & Distributed Transactions
08 Resilience & Fault Tolerance
09 Security & Observability
10 Scaling & Deployment
11 Design Patterns
12 Real-World System Design

From Beginner to System Design

Monolith → Microservices → Distributed Systems → Production Architecture

Manjunatha HG

What are Microservices ?

Introduction & Key Concepts

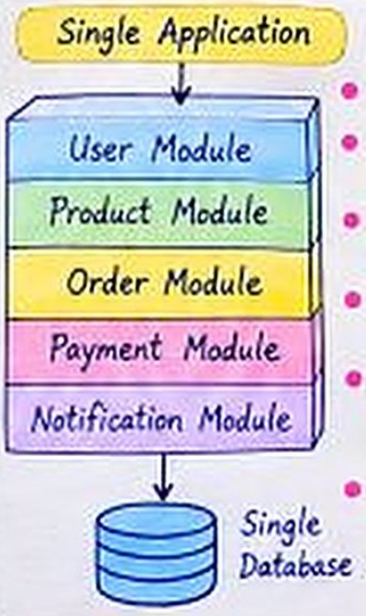
Definition

Microservices is an architectural style where an application is built as a collection of small, independent and loosely coupled services. Each service is responsible for a specific business capability and can be develeped, deployed and scaled independently.

Key Characteristics

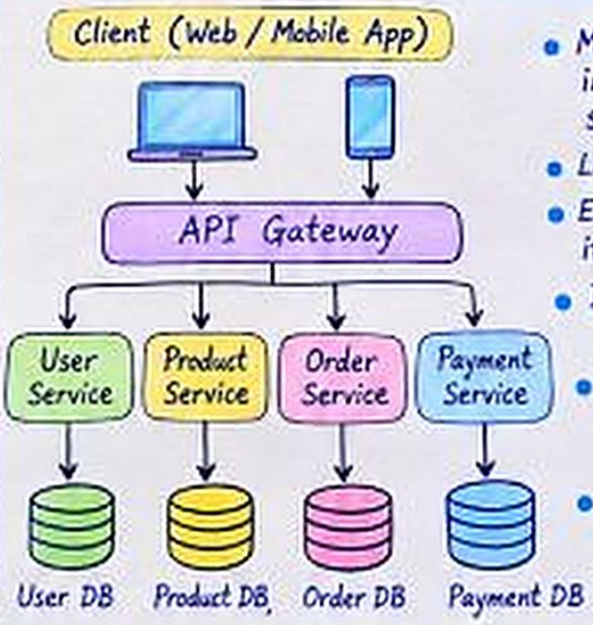
01 Independent and loosely coupled
02 Business capability based
03 Independently deployable
04 Own data / storage
05 Can be developed using different technologies
06 Communicates via lightweight mechanisms (REST, gRPC, events)

Monolith Architecture



- Single codebase
- Tightly coupled modules
- Deployed as one unit
- Usually shared database
- Scaling means scaling the entire application
- Simple to develop initially

Microservices Architecture



- Multiple independent services
- Loosely coupled
- Each service has its own database
- Independently deployable
- Can scale individual services
- More complex operationally

Real-World Example



Manjunatha HG

Why Microservices ?

Key Benefits & When Not to Use

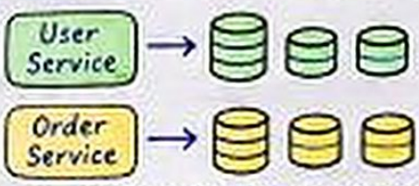
Key Benefits of Microservices

01 Independent Deployment



- Each service can be developed, deployed and released independently.
- Faster time to market.

02 Independent Scaling



- Scale only the services that need it.
- Better resource utilization.

03 Team Autonomy



- Different teams can work on different services.
- Teams can choose the best technology for their service.

04 Fault Isolation



- If one service fails, others continue to work.
- Improves overall system reliability.

05 Technology Flexibility



- Each service can be built using different technology stack.
- Use the best tool for the job.

06 Faster Development



- Smaller codebase easy to understand and maintain.
- Parallel development by multiple teams.

When NOT to Use Microservices ?

01 Small or Simple Applications



- If the application is small and straightforward.
- Microservices can add unnecessary complexity.

02 Small Teams



- Requires more operational overhead.
- A small team may struggle to manage multiple services.

03 Low Scalability Requirements



- If the application does not need to scale significantly.
- A well-designed monolith can be sufficient.

04 High Operational Complexity



- Requires expertise in DevOps, monitoring, CI/CD, service discovery, etc.
- Increases infrastructure and maintenance cost.

Manjunatha HG

Monolith vs Microservices

Key Differences & Comparison

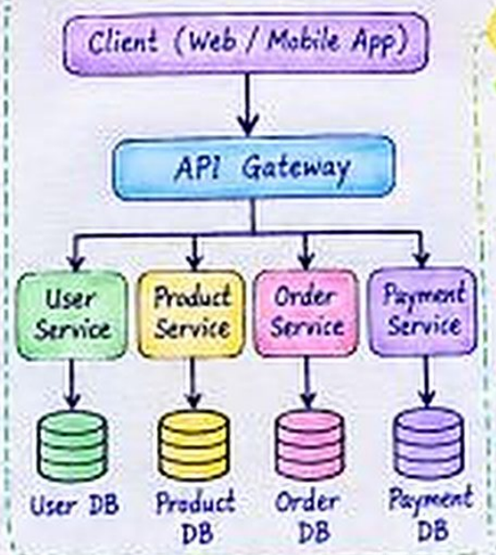
Monolith Architecture



Characteristics

- Single deployable unit.
- All modules in one codebase.
- Usually shared database.
- Tightly coupled components.
- Simple to develop and deploy (initially).
- Scaling means scaling the entire application.

Microservices Architecture

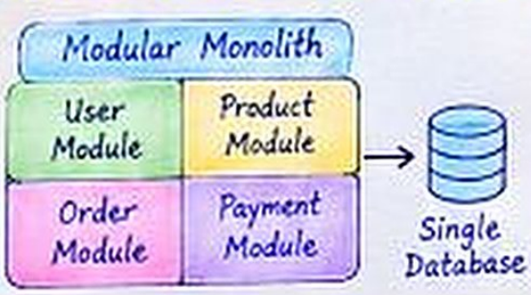


Characteristics

- Multiple independent services.
- Each service has its own database.
- Loosely coupled services.
- Independent deployment.
- Can scale individual services.
- More complex operationally.

Aspect	Monolith	Microservices
Deployment	Single deployment	Independent deployment
Codebase	Single codebase	Separate codebases
Database	Usually shared database	Database per service
Scaling	Scale entire application	Scale individual services
Coupling	Tightly coupled	Loosely coupled
Technology Stack	Single technology stack	Different technology stacks allowed
Team Structure	Single team	Multiple teams (service-wise)
Development Speed	Slower as application grows	Faster and independent development
Fault Isolation	Failure in one module can bring down entire app	Failure isolated to a single service
Operational Complexity	Relatively simple	More complex (needs DevOps, monitoring, etc.)
Best Suited For	Small to medium applications	Large, complex, highly scalable applications

Modular Monolith (Middle Ground)



What is it ?

- A monolith with well-defined modules.
- Modules are loosely coupled within the same application.
- Single deployment and usually a single database.
- Helps in better organization and can be split into microservices later if needed.

When to use ?

- Small to medium applications.
- Small teams.
- You want simplicity but with better structure.
- Can gradually migrate to microservices in the future.

Manjunatha HG

How to Identify Microservices ?

Service Boundaries & Best Practices

Key Principles

01 **Business Capability Based**
Each service should represent a specific business capability.

02 **High Cohesion**
Service should have a single, well-defined responsibility.

03 **Low Coupling**
Services should be independent and loosely coupled.

04 **Data Ownership**
Each service owns its own data and database.

05 **Independent Deployment**
Can be developed, deployed and scaled independently.

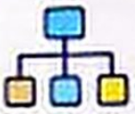
Approaches to Identify Services

Domain Driven Design (DDD)



- Use Bounded Context.
- Identify core domain areas.
- Separate services based on domains.

Business Capabilities



- Break down the application into business capabilities.
- Each capability = one service.

Subdomain Classification



- Core Domain → Separate service.
- Supporting Domain → Separate service.
- Generic Domain → Can be shared.

Questions to Ask

01 What business capability does this service represent?

02 Does it have a single responsibility ?

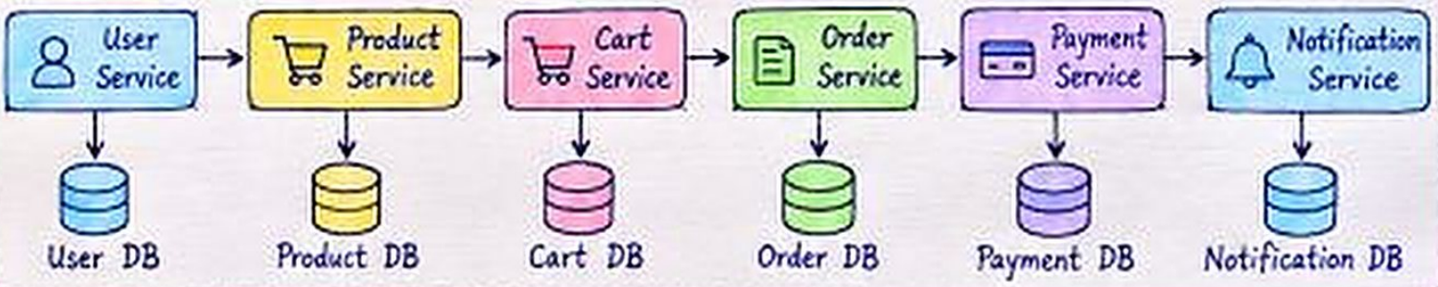
03 Can it be developed, deployed and scaled independently ?

04 Does it require its own database ?

05 Does it have minimal dependency on other services ?

06 Will it be maintained by a separate team ?

Example : E-commerce Application



Bounded Context Example



Mapping to Microservices

User Management	→ User Service
Catalog Management	→ Product Service
Order Management	→ Order Service
Payment Management	→ Payment Service
Notification Management	→ Notification Service

Best Practices

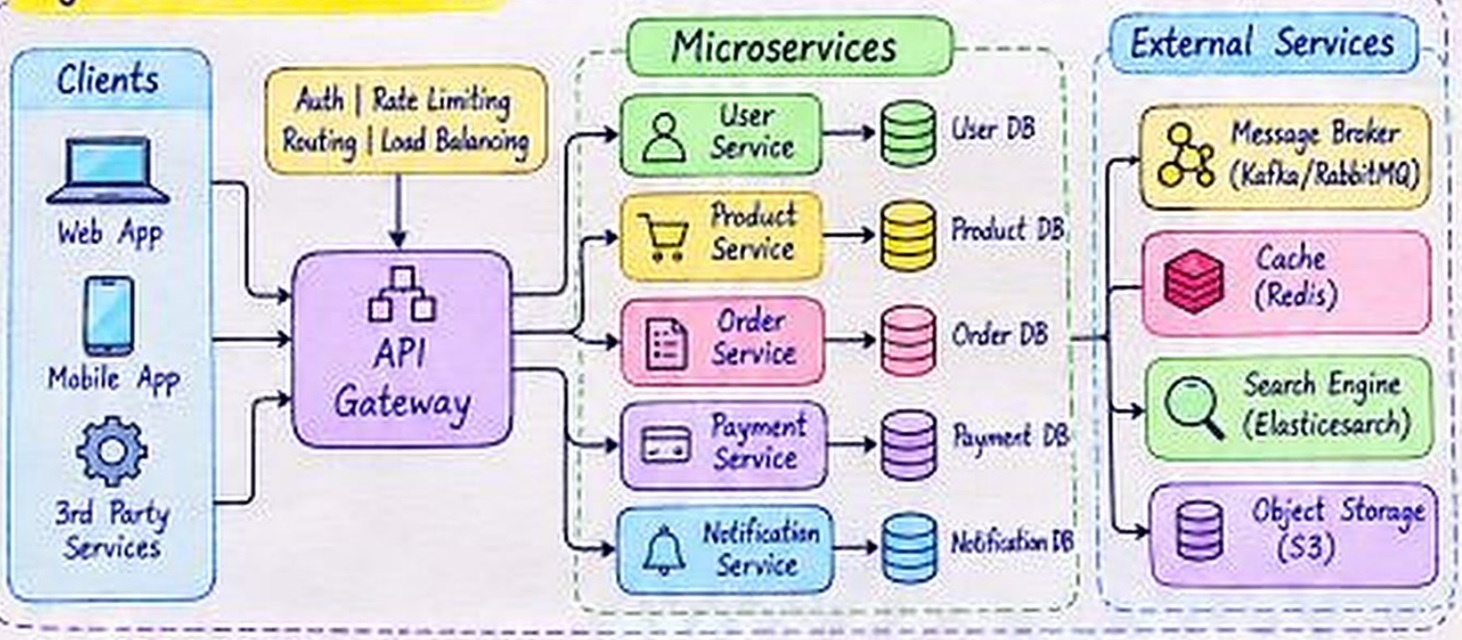
- Start with business capabilities.
- Keep services small and focused.
- Avoid shared databases.
- Use domain-driven design.
- Minimize cross-service dependencies.
- Design for independent deployment.
- Evolve services over time.

Manjunatha HG

Microservices Architecture

Overall Architecture & Components

High Level Architecture .



What is this Architecture ?

- Application is divided into small, independent services.
- Each service handles a specific business capability.
- Services communicate with each other via APIs or events.
- Each service has its own database.
- Services can be developed, deployed and scaled independently.

Key Components

01 **Client**
Web, Mobile or 3rd party app that sends requests.

02 **API Gateway**
Single entry point for all clients.

03 **Microservices**
Independent services for different business capabilities.

04 **Databases**
Each service owns its own database.

05 **External Services**
Shared infrastructure like Kafka, Redis, S3, etc.

Flow of a Request

01 Client sends request to API Gateway.

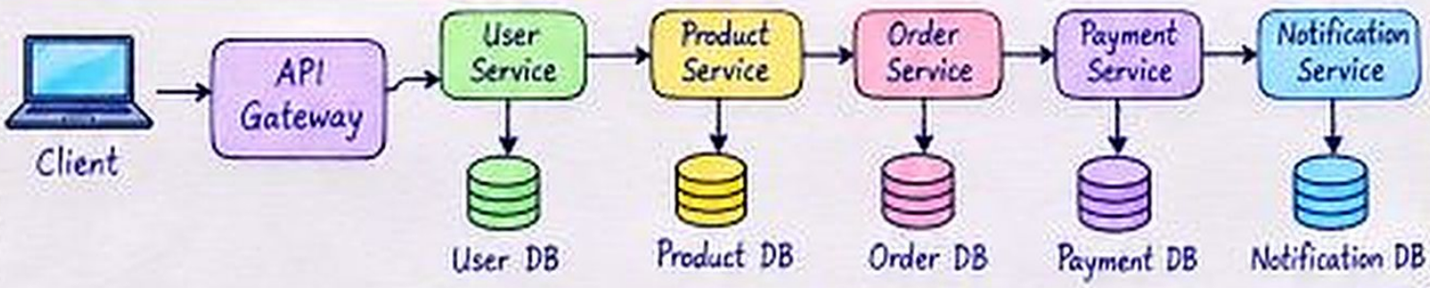
02 Gateway handles auth, routing and rate limit.

03 Request is routed to the appropriate microservice.

04 Microservice processes the request using its own database.

05 Response is returned to the client.

Example : E-commerce Application



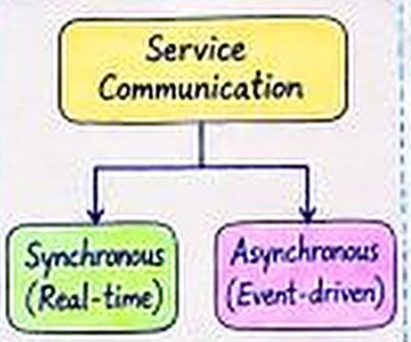
Service-to-Service Communication

How Microservices Talk to Each Other ?

Why Services Need to Communicate?

- Services are independent but still need to work together.
- One service may need data or action from another service.
- Communication can be synchronous (or asynchronous), depending on the use case.

Types of Communication



Comparison

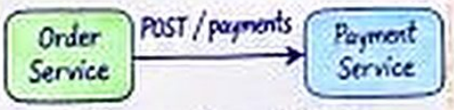
Aspect	Synchronous	Asynchronous
Nature	Request-Response	Event-based
Examples	REST, gRPC	Kafka, RabbitMQ
Coupling	Tightly coupled	Loosely coupled
Response Time	Immediate	Eventually
Use Case	Real-time data needed	Decoupled and scalable systems

Synchronous Communication



- One service makes a request and waits for the response.
- Used when immediate response is required.
- Common protocols: REST (HTTP), gRPC.
- Simple and easy to understand.

Example: Order → Payment (REST)



Response: Payment Status

Asynchronous Communication



- One service publishes an event and continues, without waiting for a response.
- Other services consume the event asynchronously.
- Used for decoupling, better scalability and fault tolerance.
- Common brokers: Kafka, RabbitMQ.
- Ideal for non-blocking operations.

Example: Order Created Event (Kafka)



When to Use Which ?

Use Synchronous (REST / gRPC) when:

- Immediate response is needed.
- Simple request-response flow.
- Low latency requirement.
- Services need real-time data.
- Example: User profile, product details.

Use Asynchronous (Kafka / RabbitMQ) when:

- Decoupled architecture is desired.
- Better scalability and fault tolerance.
- Processing can be done later.
- Handling high traffic.
- Example: Order placed, send email, generate invoice.

Communication Protocols

REST (HTTP)
- Simple, widely used.
- Uses JSON.

gRPC
- High performance.
- Uses Protocol Buffers.
- Good for internal communication.

Kafka
- High throughput.
- Event streaming.
- Used for asynchronous communication.

RabbitMQ
- Message queue.
- Reliable message delivery.
- Supports multiple consumers.

Best Practices

01. Choose the right communication type based on use case.
02. Use asynchronous communication for critical tasks.
03. Use proper error handling, retries and idempotency.
04. Define clear event schemas (for Kafka).
05. Avoid tight coupling between services.

Manjunatha HG

API Gateway

Single Entry Point for All Clients

What is API Gateway?

- API Gateway is a **single entry point** for all client requests.
- It sits between clients and **multiple microservices.**
- It handles common concerns like routing, authentication, rate limiting, etc.

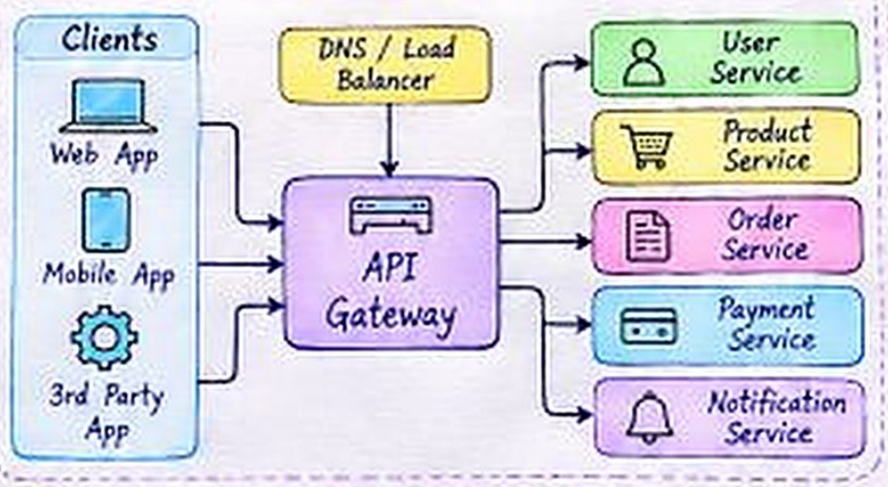
Why Do We Need It?

- Clients don't need to know multiple service URLs.
- Centralized cross-cutting concerns (auth, logging, etc.).
- Hides internal service structure.
- Improves security and control.
- Supports **request aggregation.**
- Simplifies client-side logic.

Key Responsibilities

01. Routing requests to appropriate services.
02. Authentication and authorization.
03. Rate limiting and throttling.
04. Load balancing.
05. Request/Response transformation.
06. Caching (response caching).
07. SSL termination.
08. Logging, monitoring and analytics.
09. Request aggregation (combine calls).
10. Versioning and backward compatibility.

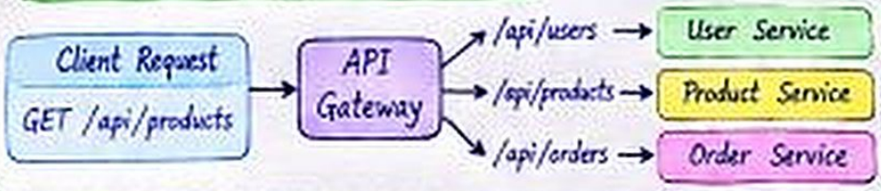
Architecture Overview



Request Flow

01. Client sends request to API Gateway.
02. Gateway authenticates the request.
03. Applies rate limiting, routing, etc.
04. Forwards request to the target service.
05. Service processes and returns response.
06. Gateway may transform/aggregate response.
07. Response is sent back to the client.

Example : Request Routing



API Gateway vs Load Balancer

Aspect	API Gateway	Load Balancer
Purpose	Application level routing	Traffic distribution
Works at	Layer 7 (Application)	Layer 4 (Transport)
Features	Auth, rate limiting, routing, transformation, etc.	Distributes traffic only
Intelligence	High (understands request)	Low (only forwards)
Use Case	Microservices, APIs	Service instances, servers

Manjunatha HG

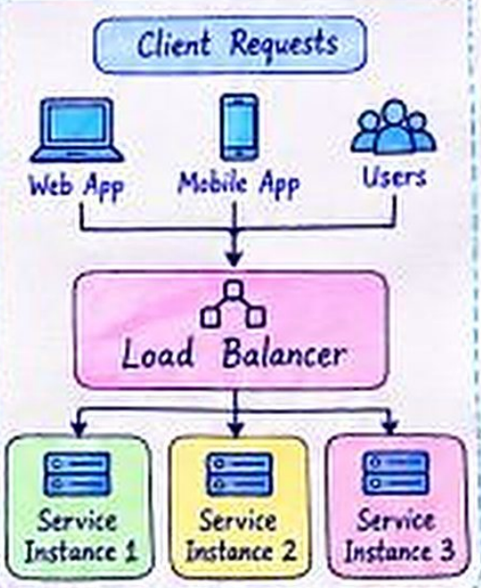
Load Balancer in Microservices

Distribute Traffic, Improve Availability & Scale

What is a Load Balancer?

- A load balancer distributes incoming traffic across multiple servers or service instances.
- It ensures high availability, fault tolerance and better resource utilization.
- In microservices, it helps to route requests to the right service instances.

How Load Balancer Works?



Why Do We Need It?

01 Distributes traffic across multiple instances.
02 Prevents single server overload.
03 Improves application availability and reliability.
04 Helps in automatic scaling (horizontal scaling).
05 Provides fault tolerance (routes traffic away from unhealthy instances).
06 Enables zero downtime deployments.
07 Handles spikes in traffic.

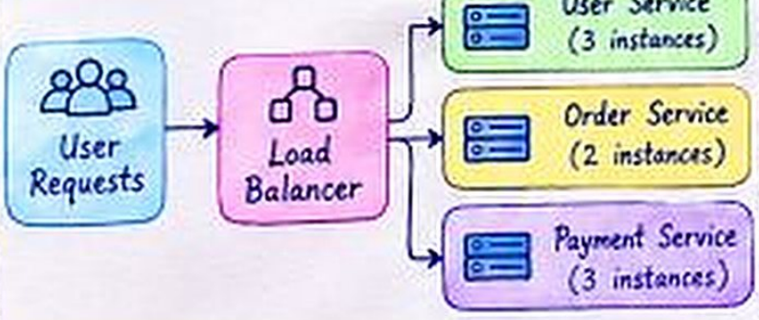
Types of Load Balancers

Type	Works At	Description	Examples
Layer 4 (Transport)	OSI Layer 4 (TCP/UDP)	Forwards traffic based on IP and port.	Nginx (TCP), AWS NLB, HAProxy
Layer 7 (Application)	OSI Layer 7 (HTTP/HTTPS)	Forwards traffic based on content (URL, headers, etc.).	Nginx, AWS ALB, Traefik, Envoy
DNS Load Balancer	DNS Layer	Routes traffic to different regions/ servers using DNS.	AWS Route 53, Cloudflare

Load Balancing Algorithms

Algorithm	How it Works	Use Case
Round Robin	Distributes requests in a fixed order.	Simple and widely used.
Least Connections	Sends request to server with least active connections.	Handles variable traffic.
IP Hash	Uses client IP to decide the server.	Keeps same user on same server (session).
Weighted Round Robin	Assigns weight to servers based on capacity.	When servers have different capacities.
Least Response Time	Sends request to server with lowest response time.	For better performance.

Example : Load Balancer in Microservices



Best Practices

- Use a Layer 7 load balancer for HTTP/HTTPS traffic.
- Enable health checks to remove unhealthy instances.
- Use auto-scaling with load balancer.
- Configure proper timeouts and retries.
- Use sticky sessions if needed.
- Implement rate limiting and DDoS protection.
- Monitor traffic, latency and error rates.
- Use multiple load balancers across regions for global availability.

Manjunatha HG

Service Discovery in Microservices

How Services Find and Communicate with Each Other ?

What is Service Discovery ?

- Service discovery is a mechanism that helps a service to find the network location (IP, port) of another service dynamically.
- Services register themselves and can discover other services without hardcoded URLs.
- It enables dynamic scaling, reduces manual configuration and improves resilience.

Why Do We Need It ?

01 Services are dynamic (IPs change frequently).

02 Supports auto-scaling and load balancing.

03 Avoids hardcoded URLs in configuration.

04 Improves fault tolerance and resilience.

05 Enables easier deployment and management.

Types of Service Discovery

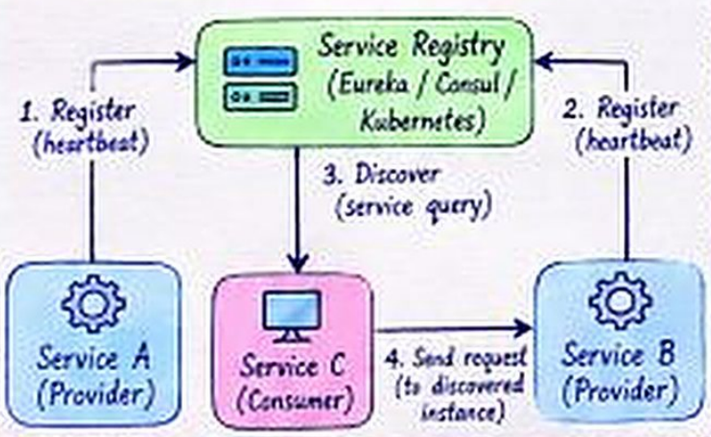
Client-side Discovery (Using Service Registry)

- Client queries the service registry to get the list of available service instances.
- Client uses a load balancing algorithm to choose a service instance.
- Examples : Netflix Eureka (Client-side), Consul.

Server-side Discovery (Using Load Balancer)

- Client sends request to a load balancer.
- Load balancer queries the service registry and forwards the request to an available service instance.
- Examples : Kubernetes Service, AWS ALB, Nginx with service registry.

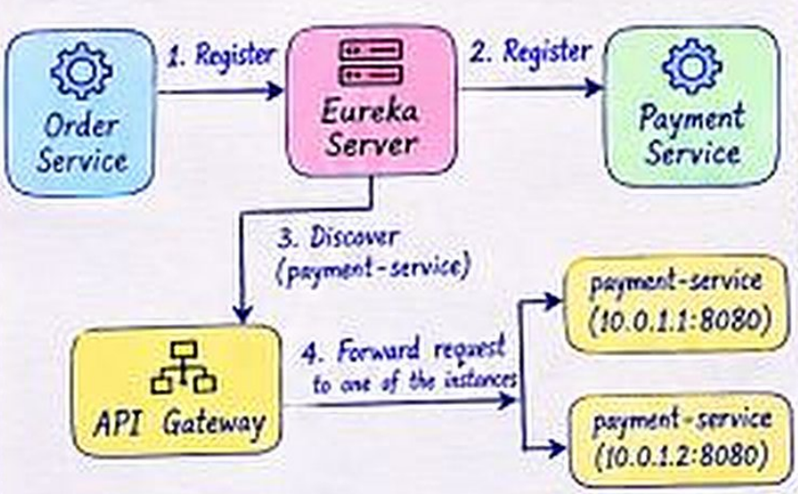
How Service Discovery Works ?



Comparison: Client-side vs Server-side

Aspect	Client-side Discovery	Server-side Discovery
Who does discovery?	Client application	Load balancer
Registry query	Client queries directly	Load balancer queries
Load balancing	Done in client	Done in load balancer
Complexity	More in client	Simpler for client
Flexibility	High (custom logic)	Less (depends on LB)
Examples	Eureka, Consul	Kubernetes Service, AWS ALB, Nginx
When to use?	When you need custom client-side load balancing	When you want a simpler setup with centralized control

Example : Using Netflix Eureka



Best Practices

- Use a reliable service registry (Eureka, Consul, Kubernetes).
- Enable health checks and automatic deregistration of unhealthy instances.
- Use TTL/heartbeat to keep the registry updated.
- Implement client-side or server-side load balancing.
- Handle service failure with retries, circuit breaker or fallback.
- Use consistent service naming and versioning.
- Secure the registry with authentication and authorization.
- Monitor service registry and discovery latency.
- Prefer Kubernetes DNS for in-cluster service discovery.

Manjunatha HG

Communication Patterns in Microservices

Different Ways Services Communicate

1 Synchronous Communication (Request-Response)

Client waits for a response from the service.



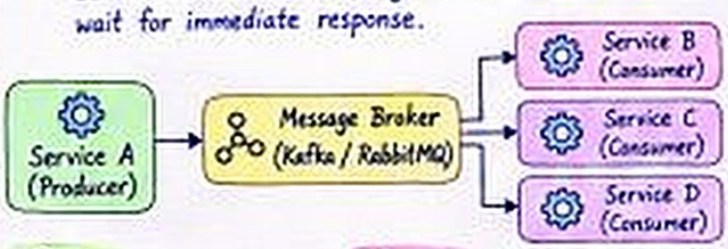
Pros :
- Simple and easy to implement.
- Real-time response.
- Good for low-latency use cases.

Cons :
- Tight coupling.
- Client waits (increases latency).
- Not suitable for long-running tasks.
- Failure in one service affects the caller.

Use Cases : User login, fetching product details, placing order (sync validation).

2 Asynchronous Communication (Event-driven)

Services communicate using events and do not wait for immediate response.



Pros :
- Loose coupling.
- Handles high traffic.
- Better scalability.
- Fault tolerant (with retries).
- Supports event-driven architecture.

Cons :
- More complex to implement.
- Eventual consistency (not real-time).
- Need to handle message failures and retries.
- Monitoring and debugging is harder.

Use Cases : Order created event, payment success event, notification event, inventory update.

3 Request-Reply using Message Queue

Client sends a request message and waits for response message.



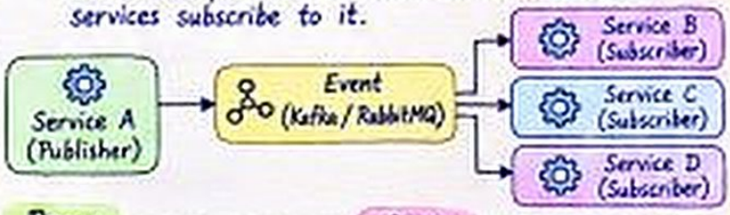
Pros :
- Decoupled and asynchronous.
- Useful for long-running tasks.
- Better fault tolerance.

Cons :
- More complex than HTTP.
- Need to manage correlation IDs.
- Higher latency compared to sync calls.

Use Cases : Report generation, video processing, Batch jobs.

4 Publish-Subscribe (Event Broadcasting)

One service publishes an event and multiple services subscribe to it.



Pros :
- One-to-many communication.
- Loose coupling.
- Easy to add new consumers.
- Scales well.

Cons :
- Eventual consistency.
- Consumers must be independent.
- Harder to track event flow.
- Message ordering is not guaranteed.

Use Cases : Order placed event → inventory, notification, analytics, recommendation services.

5 Direct Service-to-Service Call

One service directly calls another service (usually over HTTP/gRPC).



Pros :
- Simple and direct.
- Low latency.
- Easy to implement.

Cons :
- Tight coupling.
- Less fault tolerant.
- Harder to scale.

When to use ? Internal systems, simple use cases, low latency required.

6 Hybrid Model (Sync + Async)

Use synchronous calls for critical operations and asynchronous events for non-critical tasks.

Example :
- Order Service → calls Payment Service (sync).
- After payment success → publishes OrderCreated event (async).

Pros :
- Combines the best of both approaches.
- High reliability and flexibility.
- Suitable for real-world complex systems.

Cons :
- More complex design.
- Needs careful monitoring and error handling.
- Harder to maintain event flow.

Manjunatha HG

Communication Protocols in Microservices

Overview, Comparison & When to Use What ?

1 What are Communication Protocols ?

- A protocol defines the **rules** for how data is exchanged between services over a network.
- In **microservices**, different protocols are used based on use case, performance and communication pattern.
- Common protocols: HTTP/REST, gRPC, WebSocket, AMQP (RabbitMQ), Kafka and more.

2 Popular Protocols in Microservices

01 HTTP / REST
02 gRPC
03 Kafka (Event Streaming)
04 RabbitMQ (Message Queue)
05 WebSocket
06 GraphQL
07 MQTT
08 TCP / UDP

3 HTTP vs gRPC (Comparison)

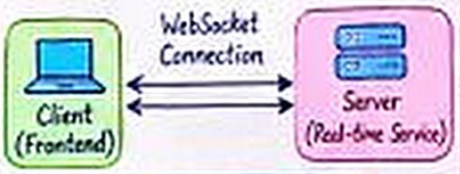
Aspect	HTTP / REST	gRPC
Protocol	HTTP (Text - JSON)	HTTP/2 (Binary - Protobuf)
Data Format	JSON	Protobuf (Binary)
Performance	Slower (text based)	Faster (binary)
Ease of Use	Simple, widely used	Complex setup
Browser Support	Native support	Limited (need gRPC-web)
Use Case	Public APIs, external clients	Internal service communication
Example	GET / orders	rpc GetOrder()

4 Kafka vs RabbitMQ (Comparison)

Aspect	Kafka	RabbitMQ
Type	Distributed log (event streaming)	Message queue
Message Model	Publish - Subscribe	Queue (Point-to-Point)
Performance	High throughput	Good (lower than Kafka)
Persistence	Yes (log based)	Yes (queue based)
Ordering	Per partition	Per queue
Use Case	Event streaming, analytics, real-time data	Task queues, asynchronous processing
Example	OrderCreated event	SendEmail job

5 WebSocket (Real-time Communication)

- Provides a **persistent, bi-directional** connection between client and server.
- Ideal for real-time use cases.

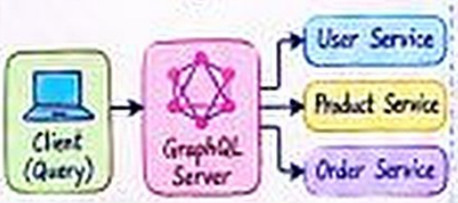


Connection stays open and both sides can send data anytime.

Use Cases : Chat applications, live notifications, stock updates, live collaboration.

6 GraphQL (Flexible API Layer)

- A query language for APIs.
- Clients can request exactly the data they need.
- Reduces over-fetching and under-fetching.



Use Cases : Complex client requirements, mobile apps, multiple frontend clients.

7 Other Protocols

MQTT	Lightweight protocol, used in IoT and low-bandwidth environments.
TCP	Reliable, connection-oriented protocol.
UDP	Faster, connectionless protocol (no guarantee of delivery).
SSE (Server Sent Events)	Unidirectional communication (server → client).

8 How to Choose ?

Use Case	Recommended Protocol
Public API (external)	HTTP / REST
Internal service-to-service communication	gRPC
Real-time communication	WebSocket or SSE
Event streaming (high volume)	Kafka
Task queues (async jobs)	RabbitMQ
Flexible client queries	GraphQL
IoT / low bandwidth	MQTT
Simple communication	HTTP / REST

9 Best Practices

- Choose the right protocol based on use case.
- Keep APIs versioned.
- Use secure communication (TLS).
- Handle retries, timeouts and failures.
- Use idempotency for retry operations.
- Monitor and log communication.
- Avoid overusing synchronous calls.
- Document APIs properly.

Manjunatha HG

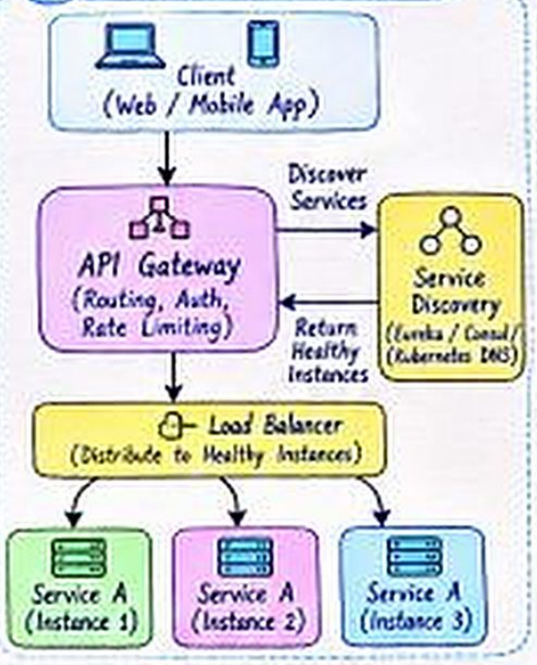
API Gateway, Load Balancer and Service Discovery Together

Complete Request Flow in a Real-World Microservices System

1 Why Do We Need All Three Together?

- API Gateway handles client requests and provides a single entry point.
- Load Balancer distributes traffic across multiple instances of a service.
- Service Discovery helps the system find the healthy instances of services automatically.
- Together, they make the system scalable, reliable, flexible and easy to manage.

2 Complete Architecture



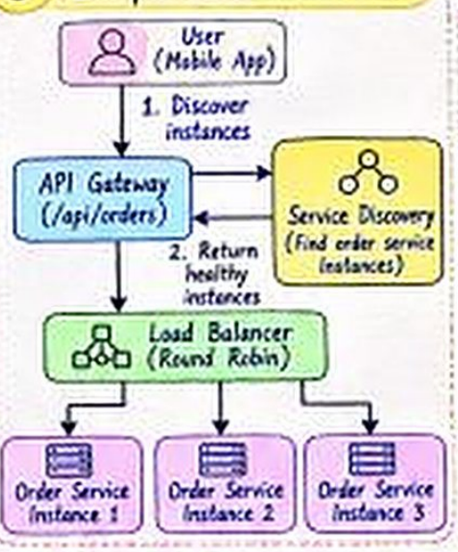
3 Request Flow Step by Step

01. Client sends request to API Gateway.
02. API Gateway authenticates the request and applies policies (rate limiting, etc.).
03. API Gateway queries Service Discovery to get healthy instances of the target service.
04. Service Discovery returns a list of available instances.
05. API Gateway forwards the request to Load Balancer.
06. Load Balancer selects a healthy instance (e.g., Round Robin).
07. Load Balancer selects to the selected service instance.
08. Service processes the request and returns the response back to the client.

4 Responsibilities Comparison

Aspect	API Gateway	Load Balancer	Service Discovery
Main Purpose	Single entry point and request management	Distribute traffic across instances	Find and register healthy service instances
Works at	Application layer (Layer 7)	Layer 4 / Layer 7	Registry / Control plane
Key Features	Routing, auth, rate limiting, request/response transformation	Load distribution, health checks, algorithms	Service registration, health checks, instance discovery
Example	Kong, NGINX, AWS API Gateway	NGINX, AWS ALB, HAProxy	Eureka, Consul, Kubernetes DNS
Handles	External client requests	Internal service traffic	Service location information

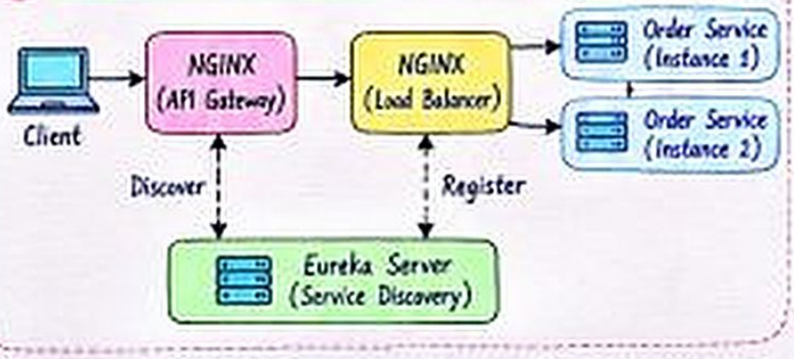
5 Example : E-commerce Flow



6 Load Balancing Algorithms

- Round Robin – simple and widely used.
- Least Connections – sends to server with least active connections.
- IP Hash – same user always goes to same server.
- Weighted Round Robin – based on capacity.
- Least Response Time – chooses the fastest server.
- Random – chooses a random server.

7 Real-World Example : Docker + Eureka + NGINX



8 Best Practices

- Use API Gateway for security, rate limiting and request routing.
- Use a reliable service discovery tool (Eureka / Consul / K8s DNS).
- Enable health checks for all services.
- Use appropriate load balancing algorithm based on use case.
- Keep services stateless for better scalability.
- Implement timeouts, retries and circuit breakers.
- Monitor services with proper logging and metrics.
- Use service versioning and gradual rollout (blue-green, canary).
- Document all endpoints and service dependencies.

Manjunatha HG

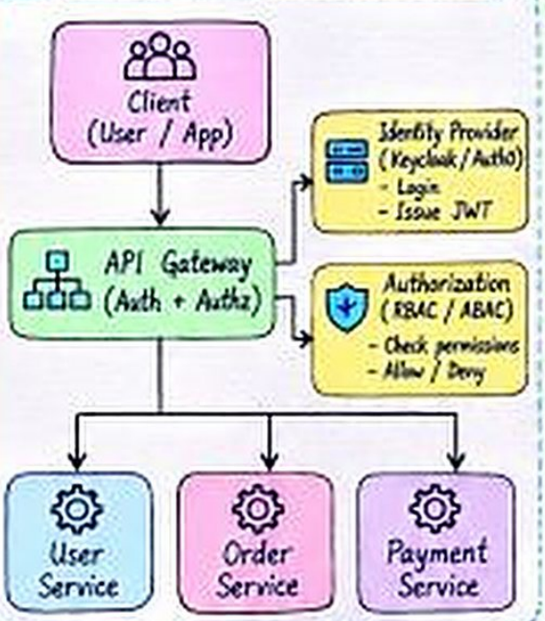
Security, Resilience and Observability in Microservices

Keep Services Secure, Reliable and Easy to Monitor

1 Security in Microservices

- Each service should be secure independently.
- Use authentication and authorization to control access.
- Use secure communication (TLS).
- Validate and sanitize all inputs.
- Use secrets management for credentials.
- Apply least privilege access.
- Implement rate limiting and throttling.
- Regularly update dependencies and patch vulnerabilities.

2 Authentication & Authorization



3 Secure Communication (TLS / mTLS)



Benefits :

- Encrypts data in transit.
- Prevents eavesdropping and tampering.
- mTLS also verifies the identity of services.
- Use certificates issued by a trusted CA.

Where to Use ?

- Service-to-service communication.
- Client to API Gateway.
- External API calls (3rd party services).

4 Common Security Measures

Measure	Description	Tools / Example
Authentication	Verify who the user is.	JWT, OAuth2, Keycloak
Authorization	Control what user can do.	RBAC, ABAC
TLS / mTLS	Encrypt and verify services.	SSL Certificates, Istio
API Security	Rate limiting, input validation.	Kong, NGINX, AWS API GW
Secrets Mgmt	Store and manage secrets.	Vault, AWS Secrets Manager
Network Security	Restrict access between services.	VPC, Security Groups
Dependency Scan	Detect vulnerable libraries.	Snyk, Dependabot
Logging & Auditing	Track access and changes.	ELK, CloudWatch

5 Resilience Patterns (Review)

Circuit Breaker	Stops calling a failing service temporarily.
Retry with Backoff	Retries failed requests with increasing delay.
Timeouts	Set time limits for requests.
Bulkhead	Isolate resources to prevent cascading failures.
Fallback	Provide default response if service fails.
Rate Limiting	Protects from traffic spikes and abuse.

6 Observability in Microservices

- Helps to monitor, debug and understand the system behavior.
- Includes logs, metrics and tracing.
- Important for identifying issues, performance bottlenecks and service dependencies.
- Use centralized tools to collect and visualize data.

7 Three Pillars of Observability

Logs

- Detailed record of events.
- Helps in debugging issues.
- Use structured logs (JSON).

Tools : ELK, Loki, CloudWatch

Metrics

- Numerical data over time (e.g., CPU, memory, latency).
- Helps in monitoring system health.
- Use real-time dashboards.

Tools : Prometheus, Grafana

Tracing

- Tracks request flow across multiple services.
- Helps in identifying latency and bottlenecks.
- Shows end-to-end request path.

Tools : Jaeger, Zipkin, AWS X-Ray

8 Best Practices

- Use authentication and authorization for all services.
- Enable TLS/mTLS for service-to-service and client communication.
- Store secrets in a secure vault (not in code).
- Implement rate limiting, circuit breakers and timeouts.
- Use centralized logging, metrics and tracing.
- Set alerts for critical system metrics.
- Monitor and audit access and configuration changes.
- Regularly update dependencies and fix vulnerabilities.
- Follow principle of least privilege.
- Test failure scenarios and run chaos engineering (e.g., Chaos Monkey).

Manjunatha HG

Emerging Trends and Future of Microservices

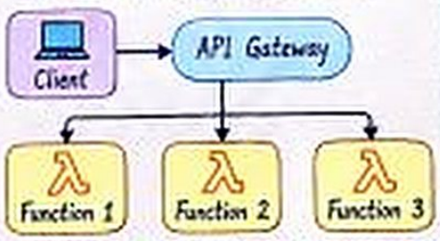
Evolution, New Technologies and What's Next ?

1 Why Microservices are Evolving ?

- Growing scale and user base.
- Need for faster feature delivery.
- Rise of cloud-native technologies.
- Demand for higher reliability, security and observability.
- New use cases like real-time data, AI/ML, edge computing.
- Businesses want more automation and self-healing systems.

2 Cloud-Native & Serverless Microservices

- Run microservices on cloud infrastructure.
- Use serverless (FaaS) for event-driven and short tasks.
- No need to manage servers.
- Auto-scaling and pay-per-use model.
- Popular platforms: AWS Lambda, Google Cloud Functions, Azure Functions.



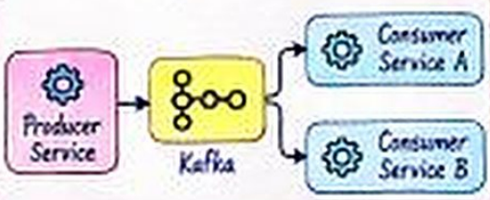
3 Service Mesh (Advanced Networking)

- Handles service-to-service communication.
- Provides traffic management, security, observability and reliability.
- Popular tools: Istio, Linkerd, Consul Connect.



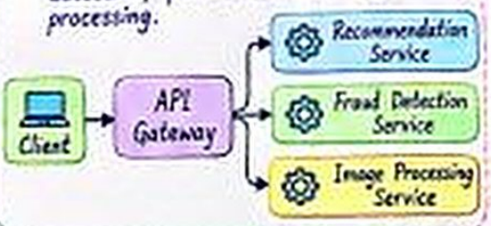
4 Event-Driven and Streaming Microservices

- Use event-driven architecture for real-time systems.
- Handle high-throughput data using streaming platforms.
- Popular tools: Kafka, Kinesis, Pulsar.
- Useful for analytics, notifications, real-time updates.



5 AI/ML in Microservices

- Microservices for AI/ML inference and training.
- Separate ML models as independent services.
- Use GPU-enabled instances for heavy workloads.
- Popular tools: TensorFlow Serving, TorchServe.
- Use cases: Recommendations, fraud detection, personalization, image/video processing.



6 Edge Computing & IoT

- Run microservices closer to users/devices.
- Reduces latency and bandwidth usage.
- Useful for IoT, AR/VR and real-time applications.
- Popular tools: AWS Greengrass, Azure IoT Edge.



7 Container Orchestration & Next Gen Tools

- Kubernetes remains the standard for orchestration.
- Newer tools simplify deployment and management.
- Popular tools: Kubernetes, OpenShift, ECS, Nomad.
- Trends: GitOps, Service Mesh, Auto-scaling.



8 Future Trends in Microservices

01. Serverless architectures and FaaS.
02. AI-powered operations (AIOps).
03. Autonomous and self-healing systems.
04. Multi-cloud and hybrid cloud deployments.
05. Better developer experience with platform engineering.
06. Increased use of event-driven and real-time systems.
07. Edge and IoT microservices.
08. Standardization and improved interoperability.
09. More focus on sustainability and cost optimization.

9 Key Takeaways

- Microservices will continue to evolve with new technologies.
- Focus on automation, observability and security.
- Leverage cloud-native and serverless when applicable.
- Use event-driven architecture for real-time use cases.
- Adopt service mesh for advanced networking.
- Keep an eye on AI/ML, edge computing and platform engineering trends.
- Always choose the right tools based on business needs.

Manjunatha HG

Real-World Case Study : E-commerce Platform

Designing a Scalable E-commerce System using Microservices

1 Requirements & Scope

Functional Requirements :

- User signup/login and profile management.
- Product catalog (search, filter, recommendations).
- Cart and wishlist.
- Order placement and tracking.
- Payments (multiple methods).
- Notifications (email, SMS, push).

Non-Functional Requirements :

- High availability and scalability.
- Low latency and high throughput.
- Fault tolerance and data consistency.
- Secure and compliant (PCI, GDPR, etc.).
- Support for high read and write traffic.

2 High-Level Architecture



3 Microservices Breakdown

Service	Responsibilities	Database
User Service	Auth, profile, addresses, user preferences.	MySQL
Product Service	Product catalog, search, filter, recommendations.	MongoDB/ Elasticsearch
Order Service	Cart, order creation, order tracking.	PostgreSQL
Payment Service	Payment processing, refunds, transaction status.	MySQL
Notification Service	Email, SMS, push notifications.	MongoDB
Inventory Service	Stock management, reservation, updates.	MySQL
Recommendation Service	Personalized product recommendations.	Redis / Elasticsearch

4 Key Workflows

1 User Places an Order



5 Product Search Flow



6 Data Management Strategy

- Each service owns its own database (Database per Service).
- Use appropriate database based on use case (Relational, NoSQL, Search, Cache).
- Eventual consistency across services.
- Use events (Kafka/RabbitMQ) for data synchronization (e.g., order → inventory updates).
- Use caching (Redis) for frequently accessed data (e.g., product details, user sessions).

7 Communication Patterns

Synchronous (Request - Response)

- Used for real-time queries.
- Examples : HTTP/REST, gRPC.
- Use cases : user login, product search, order status.

Asynchronous (Event - Driven)

- Used for long-running tasks and decoupling.
- Examples : Kafka, RabbitMQ.
- Use cases : order created event, send notification, update inventory.

8 Scaling Strategy

- Horizontal scaling (stateless services).
- Auto-scaling using Kubernetes.
- Use load balancers (NGINX / ALB).
- Caching at multiple layers (Redis, CDN).
- Database read replicas and sharding.
- Use asynchronous processing for non-critical tasks.
- Implement circuit breakers and retries.

9 Lessons Learned & Key Takeaways

- Microservices help in independent scaling and faster feature delivery.
- Use event-driven architecture for decoupling.
- Design for fault tolerance and resilience.
- Choose the right database for each service.
- Observability and monitoring are critical.
- Start simple and evolve over time.
- Focus on business requirements, not just technology.

Manjunatha HG

Monitoring, Logging, Tracing and DevOps for Microservices

Keep Microservices Healthy, Observable and Always Running

1 Why Observability ?

- Microservices have multiple services, instances and network hops.
- Hard to debug without proper observability.
- Helps in monitoring, troubleshooting, performance analysis and capacity planning.
- Includes Metrics, Logs, Traces and Alerts.

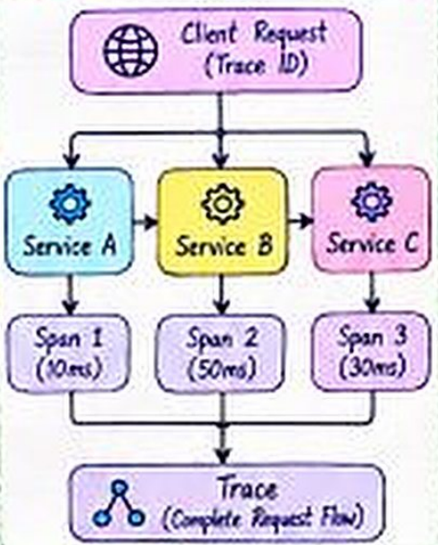
2 Three Pillars of Observability

Metrics (What is happening?)
- Numerical data (CPU, memory, latency, error rate).
- Helps in real-time monitoring.
- Tools: Prometheus, Grafana, Datadog.

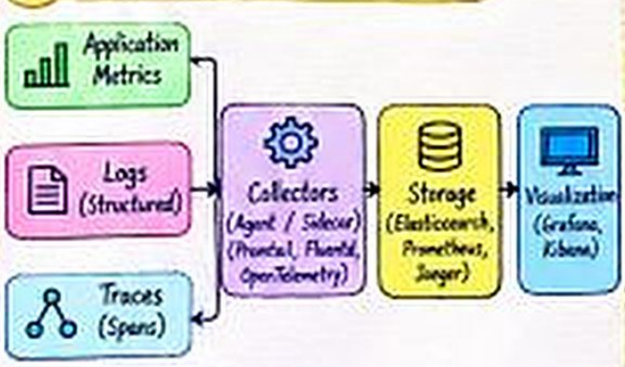
Logs (What happened?)
- Detailed event records.
- Helps in debugging issues.
- Structured format (JSON).
- Tools: ELK Stack (Elasticsearch, Logstash, Kibana), Loki.

Traces (How it happened?)
- Tracks request flow across services.
- Helps in finding latency and bottlenecks.
- Shows end-to-end transaction path.
- Tools: Jaeger, Zipkin, AWS X-Ray.

3 Distributed Tracing Flow



4 Monitoring Architecture



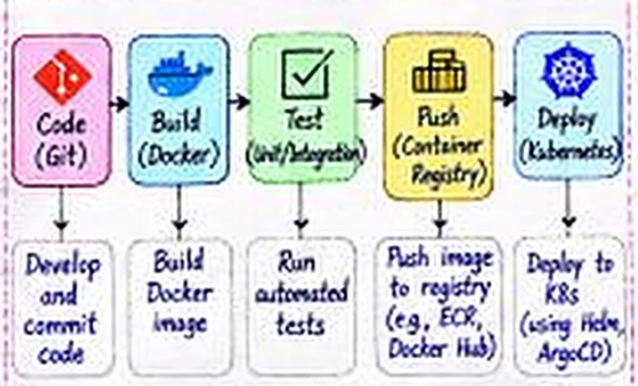
5 Common Tools in Microservices

Tool	Purpose
Prometheus	Metrics collection and alerting
Grafana	Visualization and dashboards
ELK Stack	Log management and analysis
Loki	Lightweight log aggregation
Jaeger	Distributed tracing
Zipkin	Request tracing (simple setup)
OpenTelemetry	Instrumentation and data collection
Datadog/New Relic	Full observability (metrics, logs, traces)

6 Alerting & Incident Management

- Set alerts for critical metrics (e.g., high error rate, latency, CPU usage).
- Use alerting tools: Prometheus Alertmanager, Datadog, PagerDuty.
- Define thresholds and SLAs.
- On alert, notify team via Slack, Email, etc.
- Use dashboards for quick visibility.
- Follow incident response process: Detect → Acknowledge → Investigate → Resolve → Post-mortem.

7 DevOps & CI/CD for Microservices



8 Kubernetes for Microservices

- Runs and manages containerized services.
- Auto-scaling and self-healing.
- Load balancing across pods/instances.
- Service discovery via DNS.
- Rolling updates and zero downtime.
- Use liveness and readiness probes.
- Manage configuration using ConfigMaps and Secrets.
- Popular tools: Kubernetes, Helm, ArgoCD.

9 Best Practices

- Instrument services with OpenTelemetry.
- Monitor key metrics (latency, throughput, error rate, CPU, memory).
- Centralize logs in a structured format.
- Use distributed tracing for end-to-end visibility.
- Set meaningful alerts and dashboards.
- Implement automated health checks (liveness/readiness).
- Plan for capacity and scalability.
- Conduct regular load testing.
- Perform post-mortem after incidents and learn from them.

Manjunatha HG

Resilience, Fault Tolerance and High Availability

Designing Microservices to Handle Failures Gracefully

1 Why Resilience is Important?

- Failures are inevitable (network, server, database, third-party).
- Microservices are distributed, so more points of failure.
- Resilient systems ensure better user experience.
- Prevent cascading failures.
- Maintain data consistency and availability.
- Automatically recover from transient failures.

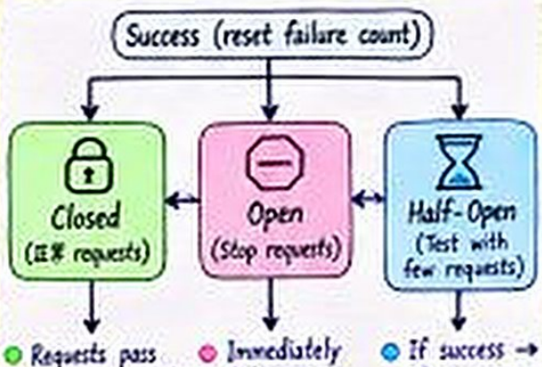
2 Common Failure Scenarios



3 Resilience Patterns

Pattern	Purpose	When to Use
Retry	Retry failed requests.	Transient failures (e.g., network issues).
Circuit Breaker	Stop calling failing service temporarily.	When a service is down/unhealthy.
Timeout	Set max wait time for a request.	Avoid hanging requests.
Bulkhead	Isolate resources per service.	Prevent one service from affecting others.
Fallback	Provide default response.	When main service is unavailable.
Rate Limiting	Control request rate.	Prevent overload.
Queueing (Message Broker)	Decouple services and handle spikes.	Handle high load and failures.

4 Circuit Breaker Pattern



- Requests pass through normally.
- Monitor failures.
- Immediately fail requests.
- Wait for cooldown period.
- If success → Close again.
- If failure → Open again.

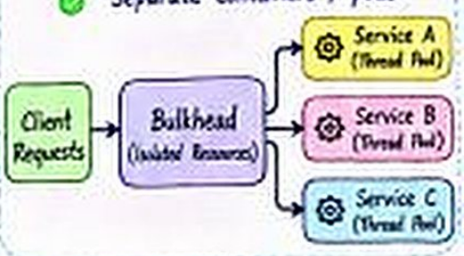
5 Retry Mechanism (with Backoff)

- Used for transient failures (network timeouts, 5xx errors).
- Use exponential backoff to avoid overwhelming the service.
- Set max retry limit.
- Add jitter to prevent thundering herd problem.

```
for (int i = 0; i < maxRetries; i++) {
    try {
        return callService();
    } catch (Exception e) {
        wait(exponentialBackoff() + jitter());
    }
}
```

6 Bulkhead Pattern

- Separate resources (threads, connection pools) for each service.
- Prevents one failing service from affecting others.
- Can be implemented using :
  - Thread pools
  - Connection pools
  - Separate containers / pods



7 High Availability Strategies

- Deploy multiple instances of each service.
- Use load balancers to distribute traffic.
- Enable auto-scaling (Kubernetes, AWS Auto Scaling).
- Use health checks and readiness probes.
- Replicate data across multiple zones/regions.
- Use message queues for asynchronous processing.
- Implement graceful shutdown and rolling deployments.

8 Graceful Degradation & Fallbacks

- When a non-critical service fails, provide a fallback response.
- Show cached data or default values.
- Disable non-essential features temporarily.
- Use feature flags to turn off failing components.
- Communicate clear error messages to users.

9 Best Practices

- Implement circuit breaker, retry and timeout.
- Use centralized logging, tracing and monitoring.
- Set proper alerts for failure rate, latency and CPU usage.
- Design for idempotency in APIs.
- Use auto-scaling and load balancing.
- Test failure scenarios (chaos engineering).
- Keep services stateless where possible.
- Document fallback behavior and SLAs.

Manjunatha HG